

SpotCheck — Xylem PDF Quality & Visual Inspection Engine

Repository: giriharan007/SpotCheck--Xylem

Primary language: Python (98.4%)

Secondary languages: Batchfile (1.3%), Shell (0.3%)

1. Executive Summary

SpotCheck is a desktop quality-assurance application for comparing an English technical-manual PDF with one or more translated PDFs. It detects missing or reordered TOC topics, broken or missing hyperlinks and cross-references, barcode and QR-code mismatches, missing/extra/moved graphics, symmetric image-count discrepancies, stylesheet-region defects, text collisions, untranslated English, margin overflows, document metadata differences, and page-level visual differences.

The system has two deliberately separated layers: `core/` contains headless PDF and image-inspection logic, while `gui/` contains the Xylem-branded desktop interface. Results are shown interactively in a comprehensive Review Tab with side-by-side inspection and exported to a 10-sheet Excel workbook with comparison evidence images.

2. Runtime Architecture

```
run_gui.py
├── gui.app_window.SpotCheckApp
│   ├── core.pipeline.run_quality_inspection()
│   ├── gui.region_marking_tab.RegionInspectorFrame
│   ├── gui.Review_tab.ComparisonGalleryFrame
│   ├── gui.Side_by_Side_preview.SideBySidePreviewWindow
│   └── gui.metadata_tab.MetadataFrame
└── core.pipeline
    ├── core.docscan          shared cached page scans /
└── multiprocessing
    ├── core.toc              TOC numerics and topic page mapping
    ├── core.links            hyperlink & cross-reference extraction
    └── and comparison
        ├── core.barcode_qr    barcode and QR validation
        ├── core.crop_images   graphic extraction and cropping
        ├── core.compare_crops visual crop matching
        ├── core.image_counts  symmetric graphic counting
        ├── core.region_engine  stylesheet-region checks
        └── core.page_diff     full-page graphic differences
```

— core.text_overlap	colliding text detection
— core.untranslated	English-left-behind detection
— core.margin_overflow	text margin boundary overflow detection
— core.metadata	document facts and column detection
— openpyxl	10-sheet Excel report generation

3. Complete Feature Inventory

3.1 Input and batch processing

- Accepts a master PDF file or a folder containing exactly one PDF.
- Accepts a translated PDF or a folder of translated PDFs.
- Validates paths and rejects ambiguous master folders.
- Creates output folders automatically beside the batch or in a custom user location.
- Remembers valid paths and the selected template between sessions.
- Runs long inspections on background threads and reports named progress stages with live percentage feedback.

3.2 TOC and translation structure

- Reads PDF bookmarks/outlines with PyMuPDF.
- Compares numeric topic identifiers such as 1, 1.1, and 3.2.1, independent of translated titles.
- Detects missing, extra, and reordered topics.
- Builds topic page spans and maps master pages to translated pages after reflow.
- Falls back to page-number matching when no usable outline exists.

3.3 Hyperlink and Cross-Reference Verification

- Extracts all URI links, web URLs, GoTo page destinations, and internal cross-references using PyMuPDF.
- Normalizes URLs, protocols (HTTP/HTTPS), trailing slashes, anchor destinations, and QR code target links.
- Performs 2D visual positional mapping (extract_topics_with_positions) to accurately bind links to their enclosing TOC topics even when multiple sub-sections share a single page.
- Uses SequenceMatcher fuzzy alignment to pair master and translated internal cross-references and detect missing or extra links.
- Gracefully handles manuals without TOC bookmarks by anchoring link occurrences directly to physical page numbers.

3.4 Barcode and QR-code checks

- Uses Pyzbar/ZBar decoding, OpenCV QR/barcode fallback detectors, and decode-independent structural detection.
- Records code type, decoded data where available, bounding box, page number, and detection method.
- Compares master and translation counts and page locations.
- Excludes barcodes and QR codes from ordinary visual image comparison because their data legitimately changes by language/document.
- Reports CHECK (No Detector) instead of falsely passing a 0/0 result when no detector is available.

3.5 Graphic extraction and comparison

- Extracts raster images, vector diagrams, logos, icons, hazard symbols, and pictures inside tables.
- Renders pages, masks text, clusters connected ink regions, and removes page furniture.
- Recognizes ruled table lattices and removes text-only table grids.
- Excludes blank regions, barcodes, QR codes, margins, thumb tabs, separators, and isolated rules.
- Groups crops by TOC topic when possible and by page otherwise.
- Uses safe Windows-compatible topic names, reserved-name handling, path-length protection, and page-folder fallback.
- Searches each master crop across translated pages with OpenCV normalized template matching.
- Uses coarse-to-fine matching, credibility thresholds, topic-scoped searches, page mapping, tie-breaking by proximity, and repeated-crop caching.
- Creates side-by-side comparison PNGs with similarity, expected page, translated page, and status.

3.6 Symmetric image counts

- Counts graphics in both documents instead of only searching for master graphics.
- Catches graphics added to translations and graphics deleted from translations.
- Compares per topic when both documents have TOCs, otherwise compares document totals.
- Uses the same graphic detector and margins as crop extraction so the two checks agree.

3.7 Stylesheet Region Inspector

- Draws, moves, resizes, deletes, and previews regions on rendered master pages.
- Supports nested regions such as Region 1.1 and automatic renumbering.
- Supports first-page, last-page, all-page, odd-page, and even-page scopes.
- Supports mirrored-layout alternative groups where any variant may pass.
- Supports exact text, visual-only, presence-only, same-pattern, scope-only, and normal layout/presence modes.
- Supports editable expected text, pinned text, token-snapped scoped exact matching, barcode/QR presence checks, per-region tolerances, and per-region thresholds.
- Uses TOC page mapping, edge-based sheet-size anchoring, per-sheet geometry, and optional scale-with-page behavior.

3.8 Text quality and margin overflow checks

- **Text overlap:** finds text spans whose actual ink collides, avoiding false positives caused only by font bounding boxes.
- **Not translated:** compares normalized translated lines against an English master inventory and uses English function-word density as supporting evidence.
- **Margin overflow:** detects translated text blocks spilling past left and right stylesheet margin thresholds (e.g., expansion during German or French localization).
- Skips first and last pages by default for text checks because covers and back matter may intentionally remain English.
- Produces evidence images and publishes findings to both the live Review tab and the Excel report.

3.9 Metadata and layout inspection

- Reports file size, pages, sheet size, orientation, dimensions, rotation, encryption, PDF version, producer, title, and TOC-entry count.
- Detects text columns using body-line projections and gutter analysis.
- Optionally excludes tables to prevent table gaps from being mistaken for document columns.
- Samples evenly distributed pages and can list every page.
- Highlights translated documents that differ from the master in page count, sheet size, or columns.

3.10 Unified Review and Side-by-Side Dual Viewer

- **Consolidated Review Tab:** Displays findings across all 10 inspection sources (TOC, Links, Barcodes, QR Codes, Images, Image Counts, Regions, Text Overlaps, Untranslated Text, Margin Overflows).
- Filters by verdict (Passed, Needs Review) with real-time issue tallies.
- Uses color-banded category tagging for clear differentiation between sub-checks.
- Scalable native treeview plus on-demand preview prevents memory exhaustion and widget bloat across hundreds of comparisons.
- **Interactive Side-by-Side Viewer (`gui/Side_by_Side_preview.py`):** Opens master and translated pages side by side with coordinate-based page anchoring.
- Features Fit Width and Fit Page zoom modes, step zooming (+ / - / 100%), and centered layout.
- Enables synchronized two-page scrolling and independent navigation without auto-scroll jumps or rendering feedback loops.
- Highlights defects with visual bounding boxes (Missing, Extra, Moved).
- Provides quick topic and page jumps with instant focus targeting.

3.11 Reporting

`core.pipeline` creates `PDF_Quality_Inspection_Report.xlsx` with 10 worksheets:

1. **Overview:** Executive dashboard of all sub-check verdicts, defect counts, run timings, and Master Verdict.
2. **TOC:** Topic numbering list, missing section codes, and section order status.
3. **Links:** Dedicated hyperlink verification sheet detailing category (URI, GoTo, QR), issue type, target destination, anchor text, master/translated pages, counts, and status.
4. **Barcode & QR:** Barcode and QR code counts, page distributions, detection methods, and status.
5. **Images:** Crop-by-crop visual comparison with page movement tracking, similarity scores, and match status.
6. **Image Counts:** Symmetric per-topic and per-document graphic counts for master and translation.
7. **Meta Data:** File size, sheet dimensions, page counts, column layouts, and sample measurements.
8. **Stylesheet Result:** Region Inspector measurements, match types, scores, vertical shifts, and verdicts.

9. **Text Overlap:** Colliding text span pairs, collision area (pt²), and affected pages.
10. **Not Translated:** English text left behind, function-word density, and verbatim matches.

4. File-by-File Inventory

File / Directory	Purpose
<code>run_gui.py</code>	Desktop entry point. Initializes frozen multiprocessing support and starts the main GUI.
<code>logger_config.py</code>	Creates timestamped logs, captures stdout/stderr/exceptions, and configures native library paths.
<code>settings.py</code>	Best-effort persistent storage for paths, template name, and margins with atomic writes and writable-directory fallback.
<code>settings.json</code>	Remembered local Windows paths, selected template, and margin settings.
<code>requirements.txt</code>	Runtime, image-processing, GUI, PDF, Excel, and packaging dependencies.
<code>check_imports.py</code>	Fast pre-build import verification for application modules and third-party packages.
<code>SpotCheck.spec</code>	PyInstaller specification with hidden imports, data files, and native binaries.
<code>build_exe.bat</code>	Windows dependency installation, import check, packaging, DLL verification, and smoke testing.

<code>build_app.sh</code>	macOS/Linux dependency installation and PyInstaller automation.
<code>core/__init__.py</code>	Documents the headless backend package and module responsibilities.
<code>core/pipeline.py</code>	Main inspection orchestrator and 10-sheet Excel report generator.
<code>core/toc.py</code>	TOC numeric extraction, comparison, topic spans, and page mapping.
<code>core/links.py</code>	Hyperlink and cross-reference extraction, URL normalization, SequenceMatcher diff, and 2D positional topic locator.
<code>core/barcode_qr.py</code>	Barcode/QR decoding, structural detection, counting, and comparison.
<code>core/docscan.py</code>	Reusable document scan cache, worker pool, page jobs, and frozen-build support.
<code>core/crop_images.py</code>	Pure graphic detection, clustering, filtering, topic grouping, and crop creation.
<code>core/compare_crops.py</code>	Cross-page graphic template matching and comparison-image creation.
<code>core/image_counts.py</code>	Symmetric per-topic/document graphic count validation.
<code>core/margins.py</code>	Margin normalization, page exemptions, content boxes, and margin classification.
<code>core/margin_overflow.py</code>	Detects translated text blocks expanding across stylesheet side margins.

<code>core/metadata.py</code>	PDF facts, sheet classification, sample-page selection, and column detection.
<code>core/page_diff.py</code>	Headless masked page comparison and missing/extra/moved graphic classification.
<code>core/templates.py</code>	JSON stylesheet-template persistence, scopes, variants, anchors, and geometry.
<code>core/region_engine.py</code>	Headless region text, presence, pattern, visual, exact, and scoped matching.
<code>core/text_overlap.py</code>	Two-stage text collision detection and evidence generation.
<code>core/untranslated.py</code>	English master inventory and untranslated-English detection.
<code>gui/__init__.py</code>	Documents the presentation-layer package.
<code>gui/theme.py</code>	Xylem palette, typography, contrast colors, and Treeview styling.
<code>gui/app_window.py</code>	Main window, tabs, path pickers, worker orchestration, live log, and cleanup safeguards.
<code>gui/region_marking_tab.py</code>	Interactive ROI editor, margin editor, stylesheet template manager, and region runner.
<code>gui/Review_tab.py</code>	Consolidated review gallery for all inspection findings (TOC, links, images, barcodes, text checks).
<code>gui/Side_by_Side_preview.py</code>	Interactive dual-page PDF viewer with Fit Width, zoom, coordinate anchoring, and difference highlighting.

<code>gui/metadata_tab.py</code>	Metadata summary and per-page display.
<code>Tables/Table_extraction.py</code>	Standalone pdfplumber table/text-box extraction and masking utility.
<code>Tables/Compare_Tables.py</code>	Standalone OCR-free table-structure comparison utility with split-table handling and Excel/JSON output.
<code>templates/10_xylem.template.json</code>	Reusable Xylem stylesheet regions, modes, anchors, margins, and page scopes.
<code>Document/implementation_plan.md</code>	Research and implementation design notes.
<code>Document/*.docx</code>	Planning, research, presentation, and project documentation artifacts.
<code>.gitignore</code>	Excludes caches, generated builds, logs, inputs/outputs, temporary files, and heavy assets.

5. Technology Stack and Why It Is Used

Technology	Use in this project	General reason for using it
Python	All application and inspection logic	Rapid development and a mature ecosystem for PDFs, image processing, GUIs, and reporting.
PyMuPDF	PDF pages, text, outlines, links, geometry, rendering, and drawing inspection	Fast, broad PDF API with high-performance vector/raster extraction and outline inspection.

OpenCV	Template matching, morphology, connected components, QR/barcode fallback, page diff	Efficient computer-vision primitives for visual comparison and shape analysis.
NumPy	Pixel masks, grayscale analysis, ink statistics, and numerical image operations	Fast vectorized array operations.
Pillow	Image conversion, resizing, previews, and composites	Simple, reliable Python image manipulation and GUI integration.
Pyzbar/ZBar	Barcode and QR decoding	Reads actual encoded data where presence-only detection is insufficient.
pdfplumber	Secondary table detection and standalone utilities	Useful PDF text/table extraction complement to PyMuPDF.
OpenPyXL	Excel report creation and formatting	Creates native 10-sheet XLSX files with rich styling without Excel automation.
CustomTkinter	Modern Xylem-branded desktop controls and tabs	Provides a modern appearance while preserving Tkinter's desktop portability.
Tkinter/ttk	Native canvases, dialogs, Treeviews, scrolling, and fallback UI	Built into Python and reliable for desktop widgets and large tables.
Threading/queues	Responsive GUI during scans and safe main-thread UI updates	Prevents long PDF operations from freezing Tkinter.

ProcessPoolExecutor	Parallel per-page scanning for long documents	Uses multiple CPU cores for expensive PDF/image analysis.
JSON	Settings and reusable templates	Human-readable, portable configuration format.
PyInstaller	Standalone executable packaging	Allows distribution to users without requiring a Python installation.

6. Running the Project

```
git clone https://github.com/giriharan007/SpotCheck--Xylem.git
cd SpotCheck--Xylem
pip install -r requirements.txt
python run_gui.py
```

Command-line execution:

```
python -m core.pipeline --english "Input\English" --translated
"Input\Translated" --output "Output"
python -m core.barcode_qr --master "<master.pdf>" --target
"Input\Translated"
python -m gui.region_marking_tab
```

Packaging:

```
python check_imports.py
.\build_exe.bat
./build_app.sh
```

7. Quality Verdict Rules

- **TOC:** Passes only when numeric topic sequences match exactly in presence and ordering.
- **Links:** Passes only when all external URLs, anchor targets, and internal cross-references match in destination and count. Reports Missing, Extra, or Mismatch otherwise.
- **Barcode/QR:** Passes when master and translated counts and locations match; structural-only detection is explicitly reported.
- **Images:** Visual crop template comparison uses an 80% pass threshold and a separate credibility floor.

- **Image Counts:** Symmetric image counts pass only when both documents contain the same counts per topic or document total.
- **Stylesheet Regions:** Verdicts depend on the selected mode (Presence, Exact Match, Visual Match, Scoped Exact) and configured tolerances.
- **Text Defects:** Text collisions (overlaps), untranslated English lines, and margin overflows lower the document verdict to FAIL.
- **Master Verdict:** Evaluates to **PASS** only when all active checks (TOC, Links, Barcodes, Images, Counts, Regions, and Text Quality) pass without defects.

8. Engineering Observations

Architecture & Pipeline Refinement: The implementation features comprehensive end-to-end coverage across structural (TOC, links), graphical (crops, counts, page diffs), and linguistic (overlaps, untranslated lines, margin overflows) dimensions. Intermediate debugging JSON dumps have been replaced with direct, high-performance in-memory pipelines connecting directly to the Review UI and the 10-sheet Excel workbook.

- The architecture cleanly separates headless inspection code (`core/`) from GUI code (`gui/`).
- Document scanning is optimized through caching and process-level parallelism.
- Visual comparison deliberately avoids comparing translated prose and language-specific code content as ordinary pixels.
- The Review Tab uses an efficient Treeview and on-demand image rendering to maintain smooth performance with hundreds of crops.
- The Side-by-Side Dual Viewer features coordinate-based page anchoring and prevents auto-scrolling during window resizing.
- Link comparison combines URL normalization, SequenceMatcher alignment, and 2D visual positional mapping to disambiguate subsections sharing pages.

9. Conclusion

SpotCheck is a specialized PDF localization and DTP quality platform rather than a simple PDF-diff script. Its main design principle is to compare the document properties that should remain invariant—structure, hyperlinks, graphics, code presence, region placement, sheet geometry, and layout—while deliberately accommodating properties that are expected to change, such as translated prose, localized barcode data, QR URLs, and pagination caused by text reflow.